

人工智慧晶片設計

期末報告

Sparse Attention RTL：Pattern Controller、K/V LineBuffer 與 QK Pair Streaming 實作

課程名稱	人工智慧晶片設計
專題題目	Sparse Attention RTL 實作與驗證
選讀論文	Flexible Yet Efficient Transformer Acceleration with Unified Sparse Attention Support on FPGA
組員	C111152325／軟體 golden model、RTL 實作、模擬驗證、報告整理
繳交日期	2026/06/14

報告摘要：本專題以 sparse attention 硬體加速為主題，完成一個可驗證的 RTL demo。實作重點放在 sparse pattern 產生、K/V line-buffer 訊號、QK dot-product 計算與 pair/MAC 統計，並以 Python golden model、pre-simulation、post-simulation 與 synthesis 報告交叉驗證。

項目	完成狀態	簡短說明
論文閱讀	完成	整理 sparse attention 對 full attention 的運算量改善，以及 pattern controller / LineBuffer 對硬體資料重用的意義。
軟體模擬	完成	使用 Python 建立 golden reference，支援 seq_len 16/32/64/128 與 full、sliding、dilated、sliding_global、butterfly 五種模式。
硬體實作	完成	以 Verilog 完成 sparse_attention_core、pattern_controller、qk_pair_streamer、KV_LBV2、qk_dot_accumulator、stats_counter 等模組。
合成	完成	完成 Synopsys Design Compiler synthesis 與 post-simulation；尚未完成實體 FPGA board 實測。

1. 論文在做什麼？

選讀論文的核心問題是 Transformer self-attention 的運算量會隨序列長度呈 $O(N^2)$ 成長。當 context window 變長時，完整 $N \times N$ attention matrix 會造成大量 QK 乘加、memory access 與 buffer 壓力，對邊緣裝置或 FPGA/ASIC 實作都不友善。

論文方向是用統一硬體支援多種 sparse attention pattern，例如 local/sliding window、dilated window、global token 與 shifted/global 類型的連線。Sparse attention 不再讓每個 token 都和所有 token 計算，而是只產生必要的 token pair，因此可以把計算量從 $O(N^2)$ 降低到與 window size 或 pattern 稀疏度相關的規模。

- 主要方法：利用 pattern controller 產生有效 Q-K pair，避免硬體計算被 mask 掉或不需要的 pair。
- 資料重用：利用 K/V LineBuffer 保存局部或 global token，降低重複讀取資料的成本。
- 硬體關聯：這類設計對 AI 晶片很重要，因為 attention bottleneck 同時牽涉 MAC 數量、記憶體頻寬、buffer 大小與 pipeline utilization。

2. 我們做了什麼？

本專題完成的是論文精神的簡化 RTL demo，目標不是做完整 Transformer accelerator，而是把 sparse attention 最關鍵的「pair generation、資料重用訊號、QK score、統計驗證」做成可跑的硬體流程。

模組 / 檔案	function
SW/attention_reference.py	Python golden model，產生 expected pair count、MAC count、checksum 與 reduction。
RTL/Rtl/pattern_controller.v	依 mode 產生 q_idx、k_idx、pair_valid 與 buffer_phase。
RTL/Rtl/qk_pair_streamer.v	整合 pattern controller 與 K/V line buffer bank，輸出 buffer_select、buffer_load、evict_valid。
RTL/Rtl/KV_LBV2.v	可參數化的 K/V line-buffer shift storage。
RTL/Rtl/qk_dot_accumulator.v	用 FEATURE_DIM 個 product lane 計算 deterministic QK score。
RTL/Rtl/sparse_attention_core.v	top wrapper，整合 pair streamer、stats counter 與 QK score pipeline。

實驗參數固定為 window_size = 4、dilation = 2、global_index = 0、feature_dim = 4，並測試 seq_len = 16、32、64、128。模式包含 full、sliding、dilated、sliding_global 與 butterfly；其中 full 作為 dense baseline，其餘模式代表不同 sparse pattern。

3. 結果如何？

軟體與 RTL 的重點驗證是：硬體產生的 pair count、cycle count 與 MAC count 是否與 Python golden reference 一致。pre-simulation 與 post-simulation 的可見測試列皆為 PASS，代表合成後 timing model 仍保留預期功能。

seq_len=128 模式	Pair count	MAC count	相對 full attention 減少
full	16,384	65,536	0.00%
sliding	512	2,048	96.875%
dilated	256	1,024	98.4375%
sliding_global	760	3,040	95.3613%
butterfly	896	3,584	94.5312%

在 seq_len = 128 時，dilated 模式的 MAC 數由 full attention 的 65,536 降到 1,024，減少 98.4375%；sliding、sliding_global 與 butterfly 也都維持 94.5% 以上的計算量降低。這符合 sparse attention 以減少 token pair 近似或取代 full attention 的設計目標。

設計	Total cell area	Max timing slack	說明
full_attention_core	24,495.179338	0.00 ns (MET)	dense baseline，完整列舉 q-k pair。
sparse_attention_core	27,704.962676	0.01 ns (MET)	加入 pattern controller、line-buffer bank、stats path 與 QK pipeline。
差異	+13.10%	皆達 10 ns 週期	Sparse 控制與 buffer 造成面積增加，但能顯著降低實際有效 MAC 數。

seq_len	mode	pairs	macs	checksum	reduction	runtime_us
16	full	256	1024	113920	0.000000	7.989
16	sliding	64	256	33600	0.750000	4.012
16	dilated	32	128	16160	0.875000	3.192
16	sliding_global	88	352	37296	0.656250	8.635
16	butterfly	64	256	31200	0.750000	2.991
32	full	1024	4096	1405952	0.000000	25.613
32	sliding	128	512	218752	0.875000	7.675
32	dilated	64	256	107072	0.937500	6.292
32	sliding_global	184	736	232752	0.820312	16.520
32	butterfly	160	640	252416	0.843750	7.000
64	full	4096	16384	19517440	0.000000	127.951
64	sliding	256	1024	1568000	0.937500	16.102
64	dilated	128	512	775296	0.968750	12.733
64	sliding_global	376	1504	1621040	0.908203	36.859
64	butterfly	384	1536	2179200	0.906250	16.385
128	full	16384	65536	289882112	0.000000	508.449
128	sliding	512	2048	11852288	0.968750	34.593
128	dilated	256	1024	5892352	0.984375	26.210
128	sliding_global	760	3040	12057136	0.953613	67.263
128	butterfly	896	3584	19347968	0.945312	36.384

圖 1. Python golden reference 輸出：列出各 seq_len 與 sparse mode 的 pair、MAC、checksum 與 reduction。

seq_len	mode	sw_pair	full_pair	sprspr_pair	full_cyc	sprspr_cyc	full_mac	sprspr_mac	reduction	check
=====										
SEQ_LEN_BLOCK 16										
=====										
RUN_BEGIN	seq_len=16 mode=full									
16	full	256	256	256	256	256	1024	1024	0.000000	PASS
RUN_BEGIN	seq_len=16 mode=sliding									
16	sliding	64	256	64	256	64	1024	256	0.750000	PASS
RUN_BEGIN	seq_len=16 mode=dilated									
16	dilated	32	256	32	256	32	1024	128	0.875000	PASS
RUN_BEGIN	seq_len=16 mode=sliding_global									
16	sliding_global	88	256	88	256	88	1024	352	0.656250	PASS
RUN_BEGIN	seq_len=16 mode=butterfly									
16	butterfly	64	256	64	256	64	1024	256	0.750000	PASS
=====										
SEQ_LEN_BLOCK 32										
=====										
RUN_BEGIN	seq_len=32 mode=full									
32	full	1024	1024	1024	1024	1024	4096	4096	0.000000	PASS
RUN_BEGIN	seq_len=32 mode=sliding									
32	sliding	128	1024	128	1024	128	4096	512	0.875000	PASS
RUN_BEGIN	seq_len=32 mode=dilated									
32	dilated	64	1024	64	1024	64	4096	256	0.937500	PASS
RUN_BEGIN	seq_len=32 mode=sliding_global									
32	sliding_global	184	1024	184	1024	184	4096	736	0.820312	PASS
RUN_BEGIN	seq_len=32 mode=butterfly									
32	butterfly	160	1024	160	1024	160	4096	640	0.843750	PASS
=====										
SEQ_LEN_BLOCK 64										
=====										
RUN_BEGIN	seq_len=64 mode=full									
64	full	4096	4096	4096	4096	4096	16384	16384	0.000000	PASS
RUN_BEGIN	seq_len=64 mode=sliding									
64	sliding	256	4096	256	4096	256	16384	1024	0.937500	PASS
RUN_BEGIN	seq_len=64 mode=dilated									
64	dilated	128	4096	128	4096	128	16384	512	0.968750	PASS
RUN_BEGIN	seq_len=64 mode=sliding_global									
64	sliding_global	376	4096	376	4096	376	16384	1504	0.908203	PASS
RUN_BEGIN	seq_len=64 mode=butterfly									
64	butterfly	384	4096	384	4096	384	16384	1536	0.906250	PASS
=====										
SEQ_LEN_BLOCK 128										
=====										
RUN_BEGIN	seq_len=128 mode=full									
128	full	16384	16384	16384	16384	16384	65536	65536	0.000000	PASS
RUN_BEGIN	seq_len=128 mode=sliding									
128	sliding	512	16384	512	16384	512	65536	2048	0.968750	PASS
RUN_BEGIN	seq_len=128 mode=dilated									
128	dilated	256	16384	256	16384	256	65536	1024	0.984375	PASS
RUN_BEGIN	seq_len=128 mode=sliding_global									
128	sliding_global	760	16384	760	16384	760	65536	3040	0.953613	PASS
RUN_BEGIN	seq_len=128 mode=butterfly									
128	butterfly	896	16384	896	16384	896	65536	3584	0.945312	PASS
=====										
TEST_PASS										
\$finish called from file "tb.v", line 470.										
\$finish at simulation time 262783000										
VCS Simulation Report										
Time: 262783000 ps										
CPU Time: 5.400 seconds; Data structure size: 2.7Mb										
Fri Jun 5 13:27:17 2026										

圖 2. Post-simulation 驗證結果：所有可見測試列皆為 PASS。

4. 遇到的問題與心得

- Sparse pattern 的困難不只在公式，而是在硬體中要逐 cycle 產生正確 q_idx/k_idx，並避免 sliding_global 等模式重複計算 local block 內已存在的 pair。
- QK score path 加入 pipeline 後，pair_valid、q_idx/k_idx 與 attention_score 需要對齊；因此 top module 以 delayed done 與 score_valid 管理輸出時序。
- LineBuffer 的 bank/select 訊號讓 demo 能呈現資料重用概念，但目前仍是簡化模型，尚未整合真實 SRAM/DRAM bandwidth 與完整 V path。

- Synthesis 後 sparse core 面積較 full baseline 增加約 13.10%，顯示 pattern control 有硬體成本；但在長序列下有效 MAC 數大幅下降，這是 sparse attention accelerator 的主要 trade-off。
- 若有更多時間，下一步會補上 softmax、SV multiplication、multi-head merge，並在 FPGA board 或更完整 ASIC flow 上量測 clock、area、power 與 throughput。

5. 參考資料

- [1] Flexible Yet Efficient Transformer Acceleration with Unified Sparse Attention Support on FPGA.
- [2] 本專題程式與報告素材：SW/attention_reference.py、RTL/Rtl/*.v、RTL/Syn/*、RES/*.png。
- [3] Synopsys Design Compiler Q-2019.12 synthesis/timing report；VCS pre-simulation 與 post-simulation 輸出。
- [4] OpenAI Codex 協助整理報告內容與 Word 版面；技術內容依本資料夾內程式、模擬輸出與合成報告撰寫。

6. README 應包含內容

Sparse Attention RTL Implementation

Team Members

- C111152325：軟體 golden model、RTL 實作、pre/post-sim、synthesis、報告整理。

What We Implemented

- Sparse attention pair generator、K/V line-buffer signals、QK dot accumulator、stats counter、testbench。

How to Run Software Simulation

- python SW/attention_reference.py

How to Run Hardware Simulation

- cd RTL/Presim；依 Presim.tcl 或 VCS command 執行 pre-simulation。
- cd RTL/Postsim；依 Postsim.tcl 搭配 synthesized netlist/SDF 執行 post-simulation。

Results and Known Issues

- Python、pre-sim、post-sim 均通過可見測試。已完成 synthesis；尚未完成 FPGA board 實測與完整 softmax/SV datapath。